
Taller de Proyecto II

Modos de Operación de los ESP

ESP8266-xx & ESP32xx

- Varios modelos de ESP8266
 - Conceptualmente similares
 - Diferencias en pines GPIO
 - Varios modelos de ESP32
 - En los últimos años (inicialmente no muchos)
 - Casi sin diferencias en lo que respecta a WiFi
 - Muchas diferencias en capacidad
 - Otras comunicaciones inalámbricas
 - Pines
 - Otros dispositivos integrados
 - Velocidades de reloj y cores
-

WiFi vs. “Servidor Web” y Conceptos

- WiFi: *asociada* a capa 1 de red - Capa física + Enlace
 - HTTP: protocolo de capa de aplicación
 - ¿Qué capa de red?
 - WiFi - AP - Router - Internet... o no
 - ¿Qué es lo más común?
 - IoT - Controladores
 - Similitudes y diferencias - Heterogeneidad
 - Caso por caso - Experiencia (o casi...)
 - Conceptos y heterogeneidades
-

Ejemplos con el ESP32

- Quizás más sencillo de programar que el ESP8266-01
 - Recordar: modelos y placas de desarrollo
 - Chip - Breakout - Placa de desarrollo
 - Más recursos (pines, RAM, Flash, ...)
 - Conceptualmente *lo mismo* para WiFi que el ESP8266
 - Sencillo para programar cosas sencillas
 - WiFi: configuración STA y AP
-

ESP STA

- Lo “básico” para funcionar como STA (station):
 - SSID y Password del AP al que se conectará
 - A partir del acceso al AP, “simplemente” tiene una IP
 - DHCP
 - Ejemplo/s de “ESP32 Dev Module”
 - En todos los casos: “server”
 - Probar con ping
 - Probar con acceso desde Internet...
-

ESP STA

```
//STAWeb
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);
...
Serial.print("\nConnected to ");
Serial.println(ssid);
Serial.print("IP address (HTTP server): ");
Serial.println(WiFi.localIP());
```

ESP STA

```
//STAWeb
```

```
WiFi.mode(WIFI_STA);  
WiFi.begin(ssid, password);
```

Esto es todo lo necesario para estar conectado a la WiFi del AP y tener una IP asignada

```
...
```

```
Serial.print("\nConnected to ");  
Serial.println(ssid);  
Serial.print("IP address (HTTP server): ");  
Serial.println(WiFi.localIP());
```

Esta es una forma sencilla aunque no muy “elegante” para tener acceso/probar las respuestas del ESP32

ESP STA

- Con WiFi.begin(ssid, password)
 - Ya se tiene IP si no hay error

```
ping 192.168.1.64
```

```
Haciendo ping a 192.168.1.64 con 32 bytes de datos:
```

```
Respuesta desde 192.168.1.64: bytes=32 tiempo=31ms TTL=255
```

```
Respuesta desde 192.168.1.64: bytes=32 tiempo=251ms TTL=255
```

```
Respuesta desde 192.168.1.64: bytes=32 tiempo=159ms TTL=255
```

¿Cómo es posible esto?

ESP STA

- Con WiFi.begin(ssid, password)
 - Ya se tiene IP si no hay error

```
ping 192.168.1.64
```

```
Haciendo ping a 192.168.1.64 con 32 bytes de datos:
```

```
Respuesta desde 192.168.1.64: bytes=32 tiempo=31ms TTL=255
```

```
Respuesta desde 192.168.1.64: bytes=32 tiempo=251ms TTL=255
```

```
Respuesta desde 192.168.1.64: bytes=32 tiempo=159ms TTL=255
```

¿Cómo es posible esto?

¡DHCP Client!

¡ping ==> ICMP!

ESP STA

- Lo anterior no tiene relación con HTTP
- La biblioteca WebServer es la que permite responder reqs.
 - No necesariamente la biblioteca “implementa” HTTP
 - Sí provee ayudas para responder reqs. HTTP

ESP STA: HTTP y Web

- Los ejemplos del ESP “mezclan” HTTP y “web”
 - HTTP es el protocolo de capa de aplicación
 - “Web” asociado a aplicaciones web...
- ¿Qué sería HTTP y qué sería “web”?
 - ¿Por qué “web” y no web?

ESP STA: HTTP y Web

- Los ejemplos del ESP “mezclan” HTTP y “web”
 - HTTP es el protocolo de capa de aplicación
 - “Web” asociado a aplicaciones web...
 - ¿Qué sería HTTP y qué sería “web”?
 - ¿Por qué “web” y no web?
 - Discriminación sencilla:
 - Si no es HTTP es “web”
 - Específico del ejemplo
-

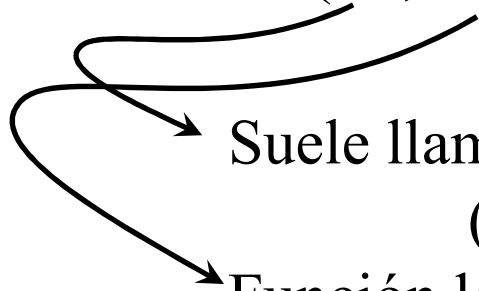
ESP STA: HTTP y Web

- De STAWeb.ino
 - `WebServer server(80);`
 - WebServer es definida/provista por la biblioteca
 - server es la variable que se usará
 - 80 es el puerto de comunicaciones (HTTP)
-

ESP STA: HTTP y Web

- De STAWeb.ino
 - `WebServer server(80);`
 - Respuestas a requerimientos (variable `server`)
 - `server.on(...)`

`server.on("/", handleRoot);`



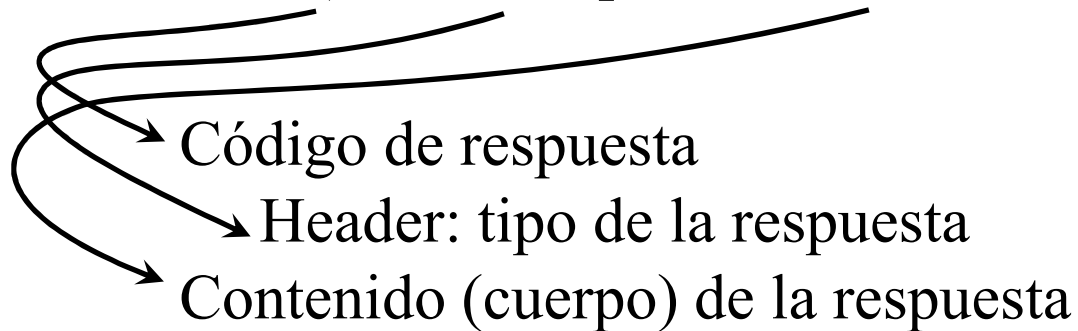
Suele llamarse o asociarse a `DocumentRoot`
(`http://<IP o DNS o ...>/`)

Función local que se encarga de la respuesta

ESP STA: HTTP y Web

- De STAWeb.ino
 - Respuesta a requerimiento (handleRoot): send
 - HTTP: request-reply

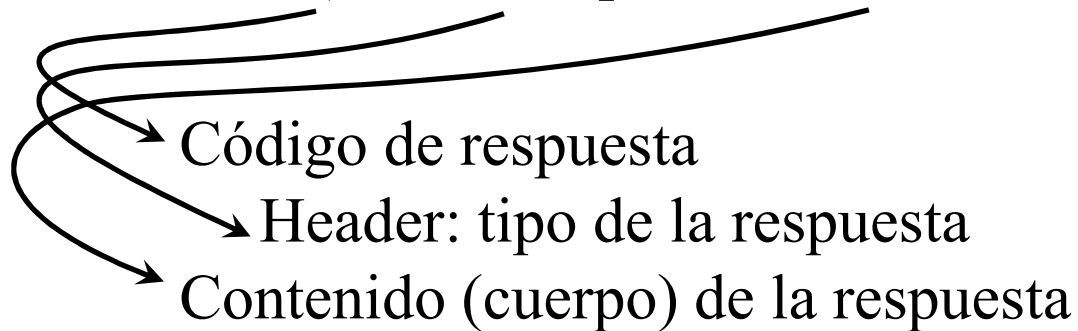
```
server.send(200, "text/plain", "Hello from Esp32!");
```



ESP STA: HTTP y Web

- De STAWeb.ino
 - Respuesta a requerimiento (handleRoot): send
 - HTTP: request-reply

```
server.send(200, "text/plain", "Hello from Esp32!");
```



¿Esto es todo lo que hay en la respuesta?

ESP STA: HTTP y Web

The screenshot shows a web browser interface. The address bar displays the URL `192.168.1.64` with a lock icon and a red 'x' icon. Below the address bar, the text "Getting Started" is visible. The main content area displays "Hello from Esp32!".

The browser's developer tools are open, showing the Network tab. The Network tab displays a list of requests:

Sta...	Me...	Do...	File	Initi...	Type	Transferr...	Size
200	GET		/	doc...	plain	101 B	17 B
404	GET		favicon	Fav...	plain	cached	16 B

The selected request (200 GET) is expanded, showing the Headers tab. The Headers tab displays the following information:

- GET**
- Scheme:** http
- Host:** 192.168.1.64
- Filename:** /

ESP STA: HTTP y Web

The screenshot shows a web browser interface. The address bar at the top displays the URL `192.168.1.64`, which is circled in black. Below the address bar, the page content reads "Hello from Esp32!".

The browser's developer tools are open, with the "Network" tab selected. The "Headers" sub-tab is active, showing the details of a GET request. The request list table is as follows:

Sta...	Me...	Do...	File	Initi...	Type	Transferr...	Size
200	GET		/	doc...	plain	101 B	17 B
404	GET		favico...	Fav...	plain	cached	16 B

The first row (200 GET) is highlighted in blue and circled in black. The "Headers" panel for this request shows the following details:

- Scheme: http
- Host: 192.168.1.64
- Filename: /

These details are also circled in black.

ESP STA: HTTP y Web

← → ↻ 🏠 192.168.1.64

🌐 Getting Started

Hello from Esp32!

🖱️ Inspector 📄 Console 🐛 Debugger ⬆️⬆️ Network {} Style Editor 🕒 Performance 🛡️

🗑️ Filter URLs || + 🔍

All HTML CSS JS XHR Fonts Images Media WS Other

Sta...	Me...	Do...	File	Initi...	Type	Transferr...	Size	Headers	Cookies	Request	Response
200	GET	📄 ...	/	doc...	plain	101 B	17 B	Filter Headers Transferred 101 B (17 B size) Request Priority Highest Response Headers (84 B) HTTP/1.1 200 OK Content-Type: text/plain Content-Length: 17 Connection: close			
404	GET	📄 ...	favicor	Fav...	plain	cached	16 B				

ESP STA: HTTP y Web

← → ↻ 🏠 192.168.1.64

🌐 Getting Started

Hello from Esp32!

🖱️ Inspector 📄 Console 🐛 Debugger ⬆️⬆️ Network {} Style Editor 🕒 Performance 🛠️

🗑️ Filter URLs || + 🔍

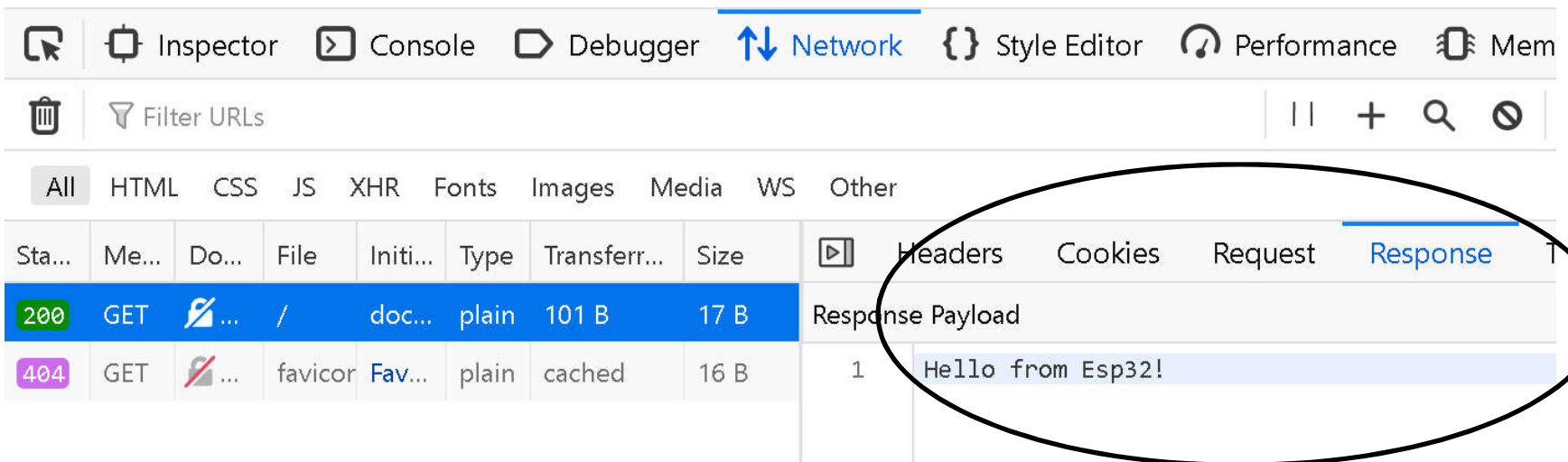
All HTML CSS JS XHR Fonts Images Media WS Other

Sta...	Me...	Do...	File	Initi...	Type	Transferr...	Size	Headers	Cookies	Request	Response
200	GET	📄 ...	/	doc...	plain	101 B	17 B	Filter Headers Transferred 101 B (17 B size) Request Priority Highest Response Headers (84 B) HTTP/1.1 200 OK Content-Type: text/plain Content-Length: 17 Connection: close			
404	GET	📄 ...	favicor	Fav...	plain	cached	16 B				

ESP STA: HTTP y Web



Hello from Esp32!



ESP STA: HTTP y Web

- Otros detalles
 - `server.onNotFound(handleNotFound)`
 - Util para responder uniformemente a todo *lo demás*
 - `server.handleClient()`
 - **No** hay atención **asincrónica**
 - Queda todo lo más elaborado por hacer
 - Nada o casi nada relacionado con la conf. como STA
-

ESP AP

- En este caso, el propio ESP es el AP
 - Es el que “maneja” la red WiFi
 - Todo lo conectado será configurado por el ESP
 - Solo relacionado con la configuración del ESP
 - Sin relación con las respuestas HTTP-web
 - Son idénticas, biblioteca WebServer
 - Ejemplo similar al anterior
 - Muy diferente al ejemplo del IDE u otros disponibles
 - Detalles
 - Primera conexión WiFi suele tardar más tiempo
 - Móviles: problemas cuando no hay clave de acceso
-

ESP AP

```
//APWeb
WiFi.mode(WIFI_AP);
WiFi.softAP(ssid, password);
...
IPAddress myIP = WiFi.softAPIP();
Serial.print("AP IP address: ");
Serial.println(myIP);
```

ESP AP

```
//APWeb
```

```
WiFi.mode(WIFI_AP);  
WiFi.softAP(ssid, password);
```

Esto es todo lo necesario para definir la WiFi del ESP (de la cual el ESP es el AP)

```
...
```

```
IPAddress myIP = WiFi.softAPIP();  
Serial.print("AP IP address: ");  
Serial.println(myIP);
```

Esta es una forma sencilla aunque no muy “elegante” para tener acceso/probar las respuestas del ESP32 (tiene una IP *default*)

ESP AP

- Con `WiFi.softAP(ssid, password)`
 - Ya se tiene AP con su propio IP si no hay error
 - Ya se tiene acceso a la red definida por el AP



ESP AP

- Con WiFi.softAP(ssid, password)
 - Ya se tiene AP con su propio IP si no hay error
 - Ya se tiene acceso a la red definida por el AP

```
ipconfig
```

```
Adaptador de LAN inalámbrica Wi-Fi:
```

```
Sufijo DNS específico para la conexión. . . :
```

```
Vínculo: dirección IPv6 local. . . : fe80::3c5f:9655:75cc:e204%13
```

```
Dirección IPv4. . . . . : 192.168.4.2
```

```
Máscara de subred . . . . . : 255.255.255.0
```

```
Puerta de enlace predeterminada . . . . . : 192.168.4.1
```

¿Cómo es posible esto?

ESP AP

- Con WiFi.softAP(ssid, password)
 - Ya se tiene AP con su propio IP si no hay error
 - Ya se tiene acceso a la red definida por el AP

```
ipconfig
```

```
Adaptador de LAN inalámbrica Wi-Fi:
```

```
Sufijo DNS específico para la conexión. . . :
```

```
Vínculo: dirección IPv6 local. . . : fe80::3c5f:9655:75cc:e204%13
```

```
Dirección IPv4. . . . . : 192.168.4.2
```

```
Máscara de subred . . . . . : 255.255.255.0
```

```
Puerta de enlace predeterminada . . . . . : 192.168.4.1
```

¿Cómo es posible esto?
¡DHCP Server!

ESP AP

- Con WiFi.softAP(ssid, password)
 - Ya se tiene AP con su propio IP si no hay error
 - Ya se tiene acceso a la red definida por el AP

```
ping 192.168.4.1
```

```
Haciendo ping a 192.168.4.1 con 32 bytes de datos:  
Respuesta desde 192.168.4.1: bytes=32 tiempo=2ms TTL=255  
Respuesta desde 192.168.4.1: bytes=32 tiempo=3ms TTL=255  
Respuesta desde 192.168.4.1: bytes=32 tiempo=2ms TTL=255  
Respuesta desde 192.168.4.1: bytes=32 tiempo=2ms TTL=255
```

¿Cómo es posible esto?

ESP AP

- Con WiFi.softAP(ssid, password)
 - Ya se tiene AP con su propio IP si no hay error
 - Ya se tiene acceso a la red definida por el AP

```
ping 192.168.4.1
```

```
Haciendo ping a 192.168.4.1 con 32 bytes de datos:  
Respuesta desde 192.168.4.1: bytes=32 tiempo=2ms TTL=255  
Respuesta desde 192.168.4.1: bytes=32 tiempo=3ms TTL=255  
Respuesta desde 192.168.4.1: bytes=32 tiempo=2ms TTL=255  
Respuesta desde 192.168.4.1: bytes=32 tiempo=2ms TTL=255
```

¿Cómo es posible esto?
¡ping ==> ICMP!

ESP AP

- Lo anterior no tiene relación con HTTP
- La biblioteca WebServer es la que permite responder reqs.
 - En este caso, idéntico al anterior



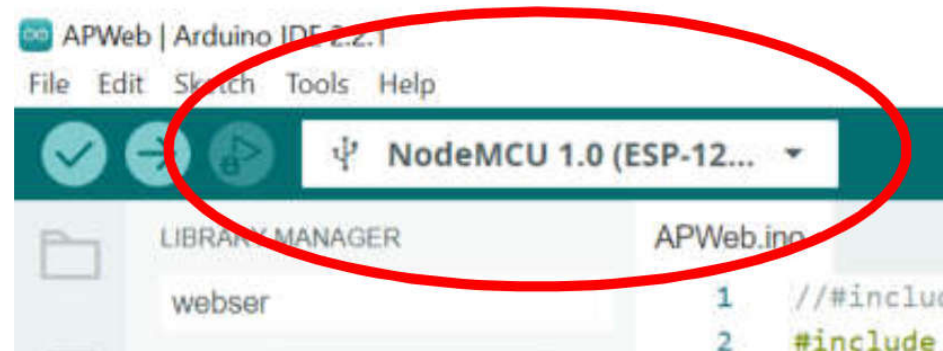
ESP AP: HTTP y Web

- Todo lo explicado para ESP EST es válido/idéntico
- Otros ejemplos difieren
 - En algunos casos “parecen otra cosa”
- Ver ejemplo del IDE de Arduino
 - WiFi ==> WiFiAccessPoint
 - Notar la diferencia y uso de “WiFiClient”



Ver Ejemplos

- ESP8266



- ESP32



SoC – Breakouts/Placas/Módulos

- ESP8266 SoC



- ESP32 SoC

